

Модуль 8

Форми. Фрейми

План

1. HTML-елементи форм
2. Валідація форм
3. Стилiзація форм
4. Фрейми

HTML-елементи форм.

Загальні поняття про форми

Веб-форми є одним з основних елементів взаємодії між користувачем та сайтом або програмою. Форми дозволяють користувачеві ввести дані, які потім відправляються на сервер для подальшої обробки та зберігання або використовуються на стороні клієнта для оновлення інтерфейсу (наприклад, додавання нового елемента до списку або відкриття та закриття елемента інтерфейсу).

Веб-форми (HTML-форми) – складаються з одного або декількох елементів управління форм (іноді їх також називають віджетами) та деяких додаткових елементів для структурування форми. Елементами управління можуть бути однорядкові або багаторядкові текстові поля, списки, кнопки, чекбокси, радіокнопки, більшість з яких створюються через html-елемент <input>, проте є й інші елементи.

Форми. Проектування та використання

В елементах управління форм можна задати **правила**, що вказують на певний **формат даних** або значень, які можуть бути введені (**валідація** форм), а також до них можуть бути додані текстові рядки, що описують ці елементи.

З точки зору зручності використання інтерфейсу, важливо пам'ятати про те, що чим довша форма, тим більший **ризик втратити користувачів**. Варто робити форму **короткою** та **лаконічною**: запитувати в користувача тільки про ту інформацію, яка дійсно необхідна.

В основному форми використовують для:

- реєстрації чи входу на сайтах;
- введення особистої інформації (ім'я, адреса, дані кредитної картки та ін.);
- організації фільтрації контенту (за допомогою списків, прапорців та ін.);
- виконання пошуку;
- завантаження файлів.

Форми. Елементи форми

HTML пропонує наступні інтерактивні **елементи** управління **форми**:

- текстові поля (для одного або кількох рядків);
- перемикачі;
- прапорці;
- списки, що випадають;
- віджети для завантаження;
- кнопки надсилання.

Ці елементи керування використовують різні теги HTML, але більшість з них використовує **тег <input>**. Оскільки цей елемент **не має закриваючого тега**, то **тип поля визначається** його **атрибутом type** :

<!-- Текстове поле -->

<input type="text">

<!-- Чекбокс (галочка) -->

<input type="checkbox">

<!-- Перемикач (точка в колі)-->

<input type="radio">

Форми. Приклад оформлення

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>HTML Forms</h2>
```

```
<form action="/my-handling-form-page" method="post">
```

```
<fieldset>
```

```
<legend> Contact data </legend>
```

```
<label for="fname"> First name: </label> <br>
```

```
<input type="text" id="fname" name="fname" value="John"> <br>
```

```
<label for="lname">Last name:</label> <br>
```

```
<input type="text" id="lname" name="lname" value="Doe"> <br> <br>
```

```
<input type="submit" value="Submit">
```

```
</fieldset>
```

```
</form>
```

<p> If you click the "Submit" button,
the form-data will be sent to a rout called
"/my-handling-form-page". </p>

```
</body>
```

```
</html>
```

HTML Forms



Contact data

First name:
John

Last name:
Doe

Submit

If you click the "Submit" button, the form-data will be sent to a rout called "/my-handling-form-page".

Елемент <form>

Тег <form> є **блоковим** елементом, який визначає **інтерактивну** частину веб-сторінки. Як результат, **всі елементи** управління (такі як <input> , <textarea> або <button>) повинні знаходитися **всередині** елемента <form> .

Для елемента <form> як правило необхідні два **атрибути** HTML:

- **action** містить адресу (чи маршрут), яка визначає, куди буде надіслана інформація форми;
- **method** (метод) в основному може бути або **GET**, або **POST** і визначає, яким чином буде надіслана інформація форми.

Як правило, інформація форми надсилається на сервер .

Елемент <form>

Форму потрібно уявляти як **набір елементів управління**, які працюють разом, **для виконання однієї операції**. Наприклад якщо потрібно написати форму входу на сайт, то варто використати три поля:

- поле для електронної пошти `<input type="email">`
- поле для пароля `<input type="password">`
- кнопка надсилання `<input type="submit">`

Ці три елементи HTML будуть укладені в одній формі

`<form action="/login" method="POST">` .

Можна аналогічно додати форму реєстрації на тій же сторінці HTML, однак вже в окремому елементі `<form>` .

Елемент <input>

Елемент HTML <input> використовується для створення інтерактивних елементів управління у веб-формах з метою отримання даних від користувача

Атрибут type задає тип елемента. Якщо цей атрибут не вказано, за замовчуванням використовується text.

Різновиди <input type"...">

Для **вводу тексту** можуть використовувати наступні значення атрибуту **type**:

email – поле для вводу адреси електронної пошти.

number – елемент управління вводу числа. Містить стрілки вгору та вниз, які дозволяють змінювати значення із сконфігурованим кроком.

password – текстове поле, чиє значення скритого точками замість символів.

search – однорядкове текстове поле для вводу рядка пошуку; розриви рядка автоматично видаляються з вхідного значення.

tel – елемент управління для вводу номера телефону;

text – однорядкове текстове поле. Переноси строки автоматично видаляються з вхідного значення.

url – поле для введення URL.

```
<label for="name">Name (4 to 8 characters):</label>  
<input type="text" id="name" name="name" required  
  minlength="4" maxlength="8" size="10">
```



Name (4 to 8 characters):

Для input type="text" підтримуються загальні атрибути

autocomplete, list, maxlength, minlength, pattern, placeholder, readonly, required та size

Кнопки

Можливими значеннями **для організації кнопок** є:

button – кнопка без наперед визначеної поведінки

reset – кнопка скидання контенту форми в стан по замовчуванню.

submit – кнопка для відсилання форми.

image – кнопка відправки у вигляді зображення. Потрібно вказати атрибут src, щоб визначити шлях до зображення та атрибут alt - для визначення альтернативного тексту. Можна використовувати атрибути height і width, щоб визначити розмір зображення в пікселях.

```
<form>
```

```
  <div class="controls">
```

```
    <label for="id">User ID:</label>
```

```
    <input type="text" id="id" name="id" />
```

```
    <input type="reset" value="Reset">
```

```
    <input type="submit" value="Submit">
```

```
  </div>
```

```
</form>
```



User ID:

Вибір `input type="checkbox"` / `radio`

Для організації вибору, як правило використовують:

checkbox – галочка («чекбокс»), прапорець – організація множинного вибору. За замовчуванням прапорець вимкнено. Ви можете помітити його увімкненням, використовуючи додатковий атрибут `checked`

`<p>Choose your monster's features:</p>`

`<div>`

`<input type="checkbox" id="scales" name="scales" checked>`
`<label for="scales">Scales</label>`

`</div>`

`<div>`

`<input type="checkbox" id="horns" name="horns">`
`<label for="horns">Horns</label>`

`</div>`

Choose your monster's features:



☒ Scales
☐ Horns

radio – кнопка-перемикач, дозволяє вибрати одне значення з множинного вибору.

`<input type="radio" id="radioButton">`



Вибір файлів `input type="file">`

Для організації вибору файлів для завантаження використовують значення атрибуту type: file – елемент управління, який дозволяє використовувати файл. Використовуйте атрибут `accept`, щоб визначити типи файлів, які можуть бути обрані.

```
<input name="myFile" type="file">
```



Вибрати файл Файл не вибрано

Приховані поля `input type="hidden"`

Для відправки певних даних на сервер, без їх демонстрації у формі використовують значення атрибуту type:

hidden – елемент управління, який не відображається, але значення відправляється на сервер.

```
<input type="hidden" id="custId" name="custId" value="3487">
```

Ввід багаторядкового тексту

Елемент `<textarea>` являє собою багаторядковий елемент керування редагуванням простого тексту, корисний, коли ви хочете дозволити користувачам вводити значну кількість тексту довільної форми, наприклад, коментар до форми відгуку або зворотного зв'язку.

```
<label for="story">Tell us your story:</label>
```

```
<textarea id="story" name="story"  
  rows="5" cols="33">
```

It was a dark and stormy night...

```
</textarea>
```



Tell us your story:

It was a dark and stormy
night...

Ввід часових параметрів

Для організації вводу часових параметрів використовують атрибути елемента `input`:

`date` – елемент управління для вводу дати (рік, місяць і день, без часу).

`datetime-local` – елемент управління для вводу дати і часу без часового поясу.

`time` – елемент управління для вводу значення часу без часового поясу.

`week` – елемент управління для введення дати, що містить число неділі та номер неділі без часового поясу.

`month` – елемент управління для вводу місяця і року без часового поясу.

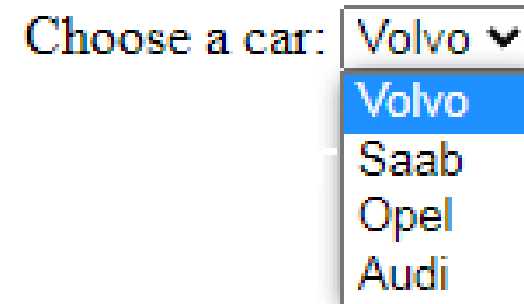
```
<label for="start">Start date:</label>  
<input type="date" id="start" name="trip-start"  
      value="2022-02-22"  
      min="2022-01-01" max="2022-12-31">
```

The diagram illustrates the visual output of the provided HTML code. On the left, a text label 'Start date:' is followed by a date input field containing '22.02.2022' and a calendar icon. A large blue arrow points from the code above to this field. To the right, the same label and field are shown, but the field is active, displaying a calendar picker. A second large blue arrow points from the input field to the calendar. The calendar is for February 2022, with the date '22' highlighted in blue. The calendar includes navigation arrows, a day-of-the-week header, and a grid of dates. At the bottom, there are buttons for 'Очистити' (Clear) and 'Сьогодні' (Today).

Випадаючий список

`<select>` (від англ. *selection* - вибір) дозволяє створити елемент інтерфейсу у вигляді списку, що розкривається, а також список з одним або множинним вибором. Кінцевий вигляд залежить від використання атрибута `size`, який встановлює висоту списку. Ширина списку визначається найширшим текстом, вказаним в елементі `<option>`, а також може змінюватись за допомогою стилів. Кожен пункт створюється за допомогою елемента `<option>`, який повинен бути вкладений у контейнер `<select>`. Якщо планується надсилати дані списку на сервер, потрібно помістити `<select>` всередину форми. Це також необхідно, коли до даних списку йде звернення через скрипти.

```
<label for="cars">Choose a car:</label>
<select id="cars" name="car" >
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="opel">Opel</option>
  <option value="audi">Audi</option>
</select>
```



Випадаючий список. Множинний вибір

Для організації множинного вибору (вибору кількох опцій) з випадаючого меню потрібно у нього додати **атрибут multiple**.

<label>Якими браузерами ви користуєтеся?</label>

<select multiple>

<option>Google Chrome</option>

<option>Internet Explorer</option>

<option>Mozilla Firefox</option>

<option>Opera</option>

<option>Safari</option>

</select>

Для того, щоб вибрати кілька варіантів потрібно утримувати зажатою клавішу Ctrl (або ⌘) та клікати мишкою. Це може бути гарною альтернативою застосуванню кількох прапорців у рядку.

placeholder

Текстові поля можуть відображати підказку(текст, який зникне, як тільки буде введено певний текст). Для цього використовують атрибут **placeholder**.

```
<input type="text" placeholder="Введіть своє ім'я">
```

Якщо тільки ви почнете щось набирати в цьому текстовому полі, то побачите, як текст «Введіть своє ім'я» **зникне**.

Особливості label

Для того, щоб підписати певне поле форми використовують мітки – тобто елемент `<label>`.

```
<input type="radio" id="kraken" name="monster">  
<label for="kraken">Kraken</label><br/>
```

Хоча й можна застосовувати інші теги (p, div та ін.) для опису елементів форми, використання `<label>` є семантично правильнішим, тому що вони існують **лише у формах**. `<label>` також може бути **пов'язана** з певним елементом форми за допомогою атрибута **for** відповідного значення **id** у поля.

```
<label for="first_name">І'мя</label>  
<input id="first_name" type="text">
```

Особливості label

При натисканні на мітку **фокус** переходить до текстового поля і **поміщає** курсор всередину нього.

Також можна розмістити **чекбокс** всередині `<label>` .

```
<label>  
  <input type="checkbox"> Я згоден з умовами  
</label>
```

Тоді можна буде клікнути текст «Я згоден з умовами», щоб переключити прапорець.

```
<label>  
  <input type="checkbox" checked> Використовувати мою платіжну адресу  
</label>
```

Особливості атрибуту name

За допомогою перемикачів можна надати користувачеві список варіантів на вибір. Для цього HTML-код повинен **згрупувати перелік перемикачів** разом. Це досягається за допомогою **одинакового** значення для атрибуту **name** (див. приклад)

Оскільки всі перемикачі використовують однакове значення атрибута name (в даному випадку значення status), вибір одного з варіантів скасовує інші. Тобто перемикачі є взаємовиключними.

Отже **атрибут name** визначає ім'я елемента. Для елементів форми він використовується як **якість назви властивості при відправці даних на сервер**, тобто без його вказання, дані на сервер будуть відправлятися НЕ КОРЕКТНО.

```
<label>Сімейний статус:</label> <br>
<label>
  <input type="radio" name="status">
    Холостяк
</label> <br>
<label>
  <input type="radio" name="status">
    Одружений
</label><br>
<label>
  <input type="radio" name="status">
    Розлучений
</label><br>
<label>
  <input type="radio" name="status">
    Вдівець
</label>
```



Сімейний статус:

- ☐ Холостяк
- ☒ Одружений
- ☐ Розлучений
- ☐ Вдівець

Поведінка браузера при натисканні кнопок в HTML-формі

Стандартна поведінка браузера при натисканні кнопок в HTML-формі залежить від типу кнопки та атрибутів форми.

Якщо кнопка має тип **submit** і знаходиться в межах форми, то при натисканні на неї відбувається відправка форми на сервер для обробки даних, якщо всі поля форми пройшли валідацію. Після відправки форми **!!! сторінка перезавантажується** зі статусом відповіді сервера.

Якщо кнопка має тип **reset** і знаходиться в межах форми, то при натисканні на неї всі поля форми очищаються і форма повертається до початкового стану.

!!! Якщо кнопка має тип **button** та не має атрибуту **type**, то при натисканні на неї браузер може сприймати її як `type="submit"` (тобто **дані форми відправляються і сторінка перезавантажується**). Для того, щоб нічого не відбувалося потрібно такій кнопці присвоїти атрибут `type="button"` (тоді можна додати власний код на JavaScript, який буде виконуватися при натисканні на кнопку).

```
<button type="button">Натискайте</button>
```

Також важливо зазначити, що **при відправці форми** на сервер, може бути виконана **валідація** на стороні клієнта та на стороні сервера, яка повертає помилки, якщо дані не пройшли валідацію.

Зміна стандартної поведінки браузера при натисканні кнопок в HTML-формі

Щоб відмінити стандартну поведінку HTML-форми при натисканні кнопки, ви можете використовувати JavaScript. При стандартній поведінці, при натисканні кнопки форма відправляється на сервер і сторінка перезавантажується. Але з JavaScript ви можете перехопити подію натискання кнопки та відмінити цю поведінку.

Для цього можна використати метод `preventDefault()` об'єкта події. Додайте обробник події для кнопки, наприклад:

У цьому прикладі, метод `preventDefault()` використовується для того, щоб відмінити стандартну поведінку кнопки. Після цього ви можете додати свій власний код, який буде виконуватися при натисканні кнопки. Наприклад, ви можете виконати якусь валідацію форми або зберегти дані локально без відправки на сервер.

```
<form>
  <button id="sbm-btn" type="submit">
    Відправити
  </button>
</form>

<script>
  document.getElementById("sbm-btn")
    .addEventListener("submit", function(event) {
    event.preventDefault();
    // Ваш код тут
  });
</script>
```


Валідація форм

Валіація форм за допомогою HTML-атрибутів

Валідація форми - це процес перевірки та підтвердження правильності введених користувачем **даних** у веб-формі **перед** їх обробкою чи **відправленням на сервер**. Мета валідації полягає в тому, щоб переконатися, що користувач ввів дані відповідно до очікуваних форматів та критеріїв.

Валідація форми може відбуватися як **на стороні клієнта** (у браузері), так і **на стороні сервера**.

Валідація **на стороні клієнта** може **негайно повідомити** користувача **про помилки** у введенні без відправки запиту на сервер, що може зменшити кількість невірних запитів, які надходять на сервер.

Всі дані повинні бути також перевірені на сервері, оскільки **валідація на стороні клієнта може бути обмеженою**. Валідація на сервері є обов'язковою для запобігання несанкціонованим або небажаним діям, оскільки клієнтська сторона може бути легко обманута або модифікована.

Валіація форм за допомогою HTML-атрибутів

Для валідації HTML-форм на стороні клієнта використовуються **атрибути**, які вказують браузерам на правила, які повинні бути дотримані при введенні даних користувачем. Ось деякі з основних атрибутів для валідації полів форм:

required - цей атрибут вказує на те, що поле є **обов'язковим для заповнення** перед тим, як форма буде відправлена (працює для елементів textarea, select, input (з типом text, password, checkbox, radio, file, datetime-local, date, month, time, week, number, email, url, search, tel);).

```
<input type="text" name="username" required>
```

Якщо ми не введемо в ці поля жодних даних, залишивши їх порожніми, і натиснемо кнопку відправки, то браузер висвітить нам повідомлення про помилку, а дані не будуть відправлені на сервер.

Валіація форм за допомогою HTML-атрибутів

Атрибут **pattern** використовується для **вказання регулярного виразу**, якому повинні відповідати дані, введені користувачем (використовується для елемента input з типом text, password, email, url, search, tel).

```
<input type="text" name="zipcode" pattern="[0-9]{5}" title="Five-digit ZIP code">
```

чи

```
<input type="text" placeholder="+38-067-370-89-01" pattern="\+\d{2}-\d{3}-\d{2}-\d{2}"  
id="phone" name="phone" />
```

Якщо ввести дані, які не відповідають цьому шаблону, та натиснути на відправку, то браузер відобразить помилку.

Валіація форм за допомогою HTML-атрибутів

Атрибути **minlength**, **maxlength** вказують мінімальну та максимальну довжину введених даних.

```
<input type="text" name="password" minlength="8" maxlength="20">
```

Атрибути **min**, **max** - встановлюють **мінімальне** та **максимальне значення** для числових полів (використовують для елемента input з типом datetime-local, date, month, time, week, number, range;).

Атрибут **step** - використовується у веб-формах **для визначення кроку** (інкременту або декременту).

```
<input type="number" name="decade" min="1950" max="2050" step="10" value="2000">
```

Стилізація форм

Стилізація форм

Вигляд HTML-форми можна значно покращити за допомогою CSS.

Властивість **width** використовується для визначення ширини поля введення:

```
input { width: 100%; }
```

Наведений вище приклад стосується всіх елементів `<input>`. Якщо потрібно створити стилі лише для певного типу `input`, то використовують селектори атрибутів:

`input[type=text]` - вибере лише текстові поля;

`input[type=password]` - вибере лише поля пароля;

`input[type=number]` - вибере лише числові поля;

Властивість **padding** використовують, щоб додати внутрішній відступ для текстового поля. Також можете додати зовнішні відступи за допомогою властивості **margin**:

```
input[type=text] {  
  width: 100%;  
  padding: 12px 20px;  
  margin: 8px 0;  
  box-sizing: border-box; }
```

Також можна встановити властивість **box-sizing** у значення **border-box**. Це гарантує, що внутрішні відступи та межі будуть включені в загальну ширину та висоту елементів.

Стилізація форм

Властивість **border** використовують для зміни розміру та кольору рамки, а властивість **border-radius** – для того, щоб додати закруглені кути:

```
input[type=text] {  
  border: 2px solid red;  
  border-radius: 4px;  
}
```

Якщо потрібна задати лише нижню межа, використовують властивість **border-bottom** :

```
input[type=text] {  
  border: none;  
  border-bottom: 2px solid red;  
}
```

За допомогою властивості **background-color** встановлюють колір фону для `<input>`, а за допомогою властивості **color** змінюють колір тексту:

```
input[type=text] {  
  background-color: #3CBC8D;  
  color: white;  
}
```


Стилізація форм

За замовчуванням деякі браузери додадуть синій контур навколо `<input>`, коли він отримує фокус (курсор поміщено в поле). Цю поведінку можете перевизначити, додавши полю властивість **outline**: none.

Селектор **:focus** використовують, щоб змінити стилізацію поля введення, коли воно отримує фокус:

```
input[type=text]:focus { border: 3px solid #555; }
```

Якщо в поле потрібно помістити піктограму (наприклад піктограму в поле пошуку), то можна використати властивості **background-image** та **background-position**. При цьому потрібно додати великий лівий відступ, щоб зарезервувати місце для піктограми:

```
input[type="search"] {  
  background-image: url('searchicon.png');  
  background-position: 10px 10px;  
  background-repeat: no-repeat;  
  padding: 12px 20px 12px 40px;  
}
```



Стилізація форм

Елементом форми можливо задати анімацію (наприклад використати властивість CSS **transition**, щоб анімувати ширину пошукового введення, коли воно отримує фокус.

```
input[type=text] { transition: width 0.4s ease-in-out; }
```

```
input[type=text]:focus { width: 100%; }
```

Також можливо стилізувати випадаючий список:

```
select {  
  width: 100%;  
  padding: 16px 20px;  
  border: none;  
  border-radius: 4px;  
  background-color: #f1f1f1;  
}
```

Стилізація форм

Для стилізації текстових областей використовують властивість **resize**, яка дозволяє запобігти зміні розміру текстових полів (вимкнути «захоплювач» у нижньому правому куті).

Властивість **resize** визначає чи може користувач змінювати розмір елемента (тобто визначає можливість і напрямок зміни розміру елемента). Не застосовується до вбудованих або блокових елементів, у яких `overflow="visible"`.

`resize: none | both | horizontal | vertical:`

Переконайтеся, що властивість **overflow** встановлено у значення «**scroll**», «**auto**» або «**hidden**».

```
textarea {  
    resize: both;  
    overflow: auto;  
}
```

Стилізація форм

Кнопки форм стилізуються аналогічно до інших кнопок веб-сторінок

```
input[type=button], input[type=submit], input[type=reset] {  
  background-color: #04AA6D;  
  border: none;  
  color: white;  
  padding: 16px 32px;  
  text-decoration: none;  
  margin: 4px 2px;  
  cursor: pointer;  
}
```

Для того, щоб встановити ширину кнопки на 100% ширини контейнера, можна задати їй властивість **width: 100%**;

Адаптивні форми

Елементи форм можливо зробити адаптивними. Для цього використовують макети сторінок на основі float, flexbox та grid, а також media-запити.

Особливості стилізації елементів форм

Потрібно зазначити, що **зовнішній вигляд** `<input type="file">` значною мірою залежать від операційної системи, під керуванням якої працює браузер, тобто є частково обмеженим в стилізації. Зазвичай для зміни його «стандарного» вигляду використовують **приховане поле** та **невидиму кнопку**, яка викликає це поле:

```
<label for="fileInput" class="custom-file-upload">  
    Custom Upload  
</label>  
<input type="file" id="fileInput" style="display:none;">
```

```
.custom-file-upload {  
    display: inline-block;  
    padding: 6px 12px;  
    cursor: pointer;  
    border: 1px solid #ccc;  
    background-color: #f8f8f8;  
}
```

Фрейми

<iframe>

Тег <iframe> визначає вбудований фрейм, який використовується для вбудовування іншого документа в поточний документ HTML. Він створює вікно (фрейм), яке може містити внутрішню сторінку з іншої адреси або файлу. Використовується для включення контенту з іншого джерела на сторінці, такого як відео, карти, віддалений контент і т. д.

```
<iframe src="https://www.example.com" width="600" height="400" frameborder="0" allowfullscreen></iframe>
```

У цьому прикладі:

src - атрибут, який вказує URL чи шлях до джерела контенту, який буде вбудований;

width, height - розміри фрейму у пікселях;

frameborder - вказує, чи має бути рамка (0 для відсутності рамки, 1 для наявності);

allowfullscreen - дозволяє включати повноекранний режим для вбудованого контенту.

Атрибути <iframe>

Тег **<iframe>** використовується для вбудовування одного документа в інший. Основні **атрибути**, які можна використовувати з тегом <iframe>, та їх можливі значення:

src – визначає URL документа, який буде завантажено в межах фрейма.

Приклад: `<iframe src="https://example.com"></iframe>`

width – встановлює ширину фрейма.

Можливі значення: будь-яке число або відсоткове значення, наприклад, **600**, **100%**.

Приклад: `<iframe width="600"></iframe>`

height – встановлює висоту фрейма.

Можливі значення: будь-яке число або відсоткове значення, наприклад, **400**, **50%**.

Приклад: `<iframe height="400"></iframe>`

frameborder – визначає наявність або відсутність рамки навколо фрейма (старий атрибут, застарілий в HTML5).

Можливі значення: **0** (без рамки), **1** (з рамкою).

Приклад: `<iframe frameborder="0"></iframe>`

Атрибути <iframe>

allow – задає дозволи на доступ до певних функцій, таких як доступ до мікрофона або камери.

Приклад: `<iframe allow="microphone; camera"></iframe>`

name – задає ім'я фрейма, яке можна використовувати для звернення до нього в JavaScript або для відкриття посилання в конкретному фреймі.

Приклад: `<iframe name="myframe"></iframe>`

sandbox – дозволяє застосувати обмеження до вмісту фрейма. Це може бути корисно для підвищення безпеки.

Можливі значення:

allow-forms – дозволяє відправку форм.

allow-scripts – дозволяє виконання скриптів.

allow-same-origin – дозволяє доступ до ресурсів з того ж джерела.

allow-popups – дозволяє відкриття спливаючих вікон.

Приклад: `<iframe sandbox="allow-scripts allow-same-origin"></iframe>`

Атрибути <iframe>

scrolling – визначає, чи можна прокручувати вміст фрейма.

Можливі значення: **yes**, **no**, **auto** (застарілий атрибут).

Приклад: `<iframe scrolling="yes"></iframe>`

referrerpolicy – визначає, яку політику передавання реферера слід використовувати при завантаженні вмісту (тобто визначає яку інформацію про попередню сторінку (реферер) браузер передаватиме при здійсненні запиту до зовнішнього ресурсу).

Можливі значення: **no-referrer**, **origin**, **unsafe-url** тощо.

Приклад: `<iframe referrerpolicy="no-referrer"></iframe>`

allowfullscreen – дозволяє фрейму відображатися на весь екран.

Можливі значення: **true** або відсутність атрибута для вимкнення.

Приклад: `<iframe allowfullscreen></iframe>`

loading – визначає, коли фрейм має завантажуватись (ліниве завантаження або завантаження при завантаженні сторінки).

Можливі значення: **lazy**, **eager**.

Приклад: `<iframe loading="lazy"></iframe>`

Ці атрибути дозволяють налаштовувати поведінку фрейма в залежності від потреб веб-сторінки.

Приклади використання <iframe>

<h1>Вмонтоване відео з YouTube</h1>

```
<iframe width="1280" height="720"
src="https://www.youtube.com/embed/i5vW9zCY3BI"
title="ТЕРНОПІЛЬ. Чим він файний? Набережна, водні прогулянки, смачна їжа та Збаразький замок"
frameborder="0"
allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share"
referrerpolicy="strict-origin-when-cross-origin" allowfullscreen></iframe>
```

<h1>Карта Google - Тернопіль</h1>

```
<iframe
src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d82835.19156362966!2d25.4785186639
4272!3d49.54870612704921!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x473036ad4b82ce75%
3A0xc484a447edb154e8!2sTernopil%2C%20Ternopil%20Oblast%2C%2046003!5e0!3m2!1sen!2sua!4v1737334
203396!5m2!1sen!2sua"
width="80%" height="450" style="border:0;" allowfullscreen="" loading="lazy"
referrerpolicy="no-referrer-when-downgrade">
</iframe>
```

<iframe>. Застереження та безпека

Використання тегу <iframe> може бути зручним для вбудовування зовнішнього контенту, але він також несе певні **ризики**, якщо його неправильно застосовувати або не враховувати безпеку. Ось кілька основних небезпек при використанні <iframe>:

1. Clickjacking - зломисник може використати <iframe> для вставлення фрейму з іншого вебсайту, який може перекривати візуальну частину сторінки, змушуючи користувача взаємодіяти з елементами, які виглядають зовсім по-іншому, ніж вони є насправді (наприклад, кнопка, що виконує іншу дію).

Для захисту використовуються HTTP заголовки, такі як **X-Frame-Options** або директиви **Content-Security-Policy (CSP)** для запобігання вбудовування вашого сайту в інші сторінки.

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' https://trusted-cdn.com; object-src 'none';">
```

default-src 'self': дозволяє завантажувати всі ресурси лише з того ж домену.

script-src 'self' https://trusted-cdn.com: дозволяє завантажувати скрипти лише з того ж домену та з <https://trusted-cdn.com>.

object-src 'none': забороняє завантаження будь-яких об'єктів, таких як Flash або Java applets.

```
<meta http-equiv="X-Content-Type-Options" content="nosniff">
```

nosniff: інструктує браузер не намагатися "відгадати" тип контенту файлу (запобігає атакам через неправильні типи контенту).

<iframe>. Застереження та безпека

<meta http-equiv="X-Frame-Options" content="SAMEORIGIN">

SAMEORIGIN: дозволяє вбудовувати сторінку лише на сайтах того ж домену.

<meta name="referrer" content="no-referrer">

no-referrer: браузер не буде передавати інформацію про те, з якого сайту користувач прийшов.

<meta http-equiv="X-XSS-Protection" content="1; mode=block">

1; mode=block: включає захист від XSS і блокує завантаження сторінки, якщо виявлено XSS

<meta http-equiv="Permissions-Policy" content="geolocation=(), microphone=(), camera=()">

відключення доступу до геолокації, мікрофону і камери на всіх сторінках вашого сайту

2. Cross-Site Scripting (XSS):

Опис: Якщо зовнішній контент у фреймі включає небезпечний JavaScript, він може виконувати шкідливі скрипти на вашій сторінці, що призводить до викрадення даних або компрометації безпеки.

Для запобігання використовують атрибут sandbox на <iframe> для обмеження дозволів, таких як виконання скриптів, взаємодія з іншими частинами сайту тощо.

<iframe src="https://example.com" sandbox="allow-scripts allow-same-origin"></iframe>

<iframe>. Застереження та безпека

3. Блокування вмісту через політику безпеки (CORS).

Деякі сайти можуть блокувати вбудовування свого контенту в інші сторінки через політику CORS (Cross-Origin Resource Sharing). Це може обмежити функціональність вашої сторінки, якщо ви намагаєтеся вбудувати ресурси, які не підтримують вбудовування.

Міри: використання налаштувань CORS на сервері для дозволу доступу лише для певних доменів.

4. Потенційний вплив на продуктивність:

Вставлені фрейми можуть уповільнити завантаження сторінки, оскільки для завантаження контенту з зовнішніх джерел потрібен додатковий запит. Якщо вбудований контент важкий або завантажується повільно, це може вплинути на користувацький досвід.

Захист: Використання атрибута `loading="lazy"`, щоб відкласти завантаження фрейма до моменту його необхідності для покращення продуктивності.

```
<iframe src="https://example.com" loading="lazy"></iframe>
```

5. Безпека вмісту в сторонніх фреймах:

Якщо ви вставляєте фрейм з неперевірених або ненадійних джерел, є ризик того, що вміст може містити шкідливий код або навіть фішинг.

Захист: Встановлення політики безпеки для вбудовування контенту лише з надійних джерел і використання атрибута `sandbox` для обмеження прав сторонніх вмістів.

<iframe>. Застереження та безпека

6. Можливі проблеми з конфіденційністю:

Веб-сайти, які вбудовують контент через <iframe>, можуть передавати інформацію про користувачів зовнішнім ресурсам. Це може бути проблемою, якщо цей контент збирає дані без згоди користувачів.

Захист: Переконайтеся, що використовується політика конфіденційності і що користувачі погоджуються з передачею своїх даних (наприклад, через cookie або інші методи відстеження).

Поради для безпечного використання <iframe>:

- Завжди застосовуйте атрибут `sandbox` для обмеження дозволів.
- Використовуйте заголовки безпеки (X-Frame-Options, Content-Security-Policy) для захисту вашого контенту від небажаних вбудовувань.
- Обмежте доступ до вашого контенту за допомогою CORS.
- Вставляйте контент лише з перевірених та надійних джерел.
- Регулярно перевіряйте безпеку зовнішніх ресурсів, якщо вони використовуються на вашій сторінці.

Ці заходи допоможуть вам **мінімізувати ризики** при використанні <iframe>.

Підсумок

1. Вивчили HTML-елементи форм
2. Вивчили принципи валідації форм на стороні клієнта
3. Навчилися стилізувати форми
4. Отримали поняття про фрейми та принципи їх використання

Як вам форми та фрейми?